PRML Spring 2026 Homework 1

Yifan Ding
Code: `https://github.com/your-username/PRML-Spring-2026-HW1`
Email: 23373493@buaa.edu.cn

March 24, 2026

Abstract

Add your abstract here.

1 Introduction

Regression analysis is a cornerstone of predictive modeling and statistical learning, widely applied in fields such as pattern recognition, computer vision, and data mining. The core challenge of regression lies in finding a robust mapping from input features to continuous output values, where the choice of model and optimization algorithm directly impacts prediction accuracy and generalization ability.

1.1 Problem Statement

Given a set of 2D data: Data4Regression, where the first sheet is training data and the second sheet is testing data.

1. Use least squares, gradient descent (GD), and Newton's method to perform linear fitting on the data, and observe their training and testing errors.
2. If you find that the linear model does not fit well (the data is actually nonlinear, so it is normal that the experimental results of the previous question are not good), can you find a more suitable model to fit the given data? Please give the reason for choosing the model, specific experimental results, and analysis of the results.

1.2 Problem Analysis

Firstly, let's take a look at the distribution of the data.

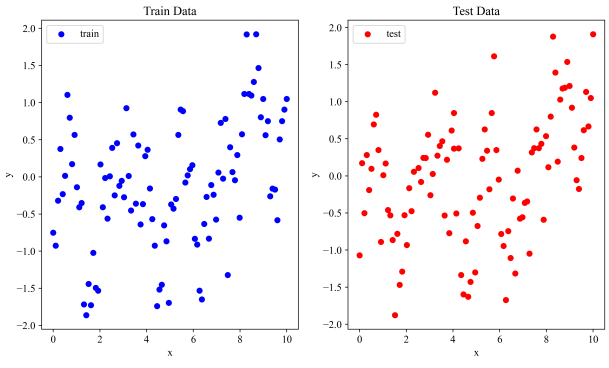

Figure 1: Distribution of given training and testing data.

As shown in Figure 1, the data is not linearly distributed, so we can expect that the linear regression model will not work well. In such cases, we should consider using a nonlinear regression model to capture the underlying patterns in the data.

Here are some potential nonlinear regression models that we can consider:

- Polynomial Regression: This model fits a polynomial function to the data, allowing for more flexibility in capturing nonlinear relationships.
- Support Vector Regression (SVR): This model uses a kernel function to map the input data into a higher-dimensional space, where a linear regression can be performed.
- Decision Tree Regression: This model uses a tree-like structure to make predictions based on the input features, allowing for complex nonlinear relationships to be captured.
- Neural Networks(NNs): Supported by Universal Approximation Theorem, neural networks with multiple layers of interconnected nodes can learn complex nonlinear relationships between the input features and the output values. Training a neural network can be computationally expensive and may require a large amount of data.

Due to the limited amount of data(100 for train and 100 for test), we will focus on polynomial regression and support vector regression in our experiments.

1.3 Structure of the Paper

The rest of this paper is organized as follows: In Section 2, we will discuss the methodology used to evaluate the performance of different models, including the metrics and algorithms for linear regression. In Section 3, we will present the experimental results of applying least squares, gradient descent, and Newton's method to the given data, and analyze the results. Finally, we will conclude the paper in Section 4.

2 Methodology

2.1 Metrics

To evaluate the performance of different models and algorithms, we can use two common metrics: root mean squared error (RMSE) and coefficient of determination (R^2). The RMSE is defined as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{1}$$

where y_i is the true value and $\hat{y}_i$ is the predicted value. The RMSE measures the average squared difference between the true values and the predicted values, and a lower RMSE indicates a better fit of the model to the data.

The coefficient of determination, denoted as R^2, is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \tag{2}$$

where $\bar{y}$ is the mean of the true values. The R^2 value ranges from 0 to 1, where a value closer to 1 indicates that the model explains a larger proportion of the variance in the dependent variable.

2.2 Baseline Model: Linear Regression

Linear regression is a fundamental statistical method used to model the relationship between a dependent variable and one or more independent variables. The goal of linear regression is to find the best-fitting line

(or hyperplane in higher dimensions) that minimizes the sum of squared differences between the observed values and the predicted values. The linear regression model can be expressed as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p, \tag{3}$$

where y is the dependent variable, $x_1, x_2, \ldots, x_p$ are the independent variables, β_0 is the intercept, $\beta_1, \beta_2, \ldots, \beta_p$ are the coefficients.

Here we will discuss three optimal algorithms for fitting a linear regression model: least squares, gradient descent, and Newton's method.

2.2.1 Least Squares

The least squares method is a mathematical approach used to find the best-fitting line (or hyperplane in higher dimensions) in linear regression by minimizing the sum of squared differences between the observed values and the predicted values. Given a set of training data points (x_i, y_i) for $i = 1, 2, \ldots, n$, the least squares method aims to find the coefficients $\beta_0, \beta_1, \ldots, \beta_p$ that minimize the cost function:

$$J(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \tag{4}$$

where $\hat{y}_i$ is the predicted value for the i-th data point, given by the linear model:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}. \tag{5}$$

The least squares method can be solved analytically using the normal equation:

$$\beta = (X^T X)^{-1} X^T y, \tag{6}$$

where X is the matrix of input features and y is the vector of observed values. This method provides a closed-form solution for the coefficients, making it computationally efficient for small datasets. However, for larger datasets or when the number of features is high, it may be computationalally expensive due to the matrix inversion step, which has a time complexity of $O(n^3)$, where n is the number of data points.

2.2.2 Gradient Descent

Gradient descent(GD) is an optimization algorithm used to minimize the cost function in machine learning models, including linear regression. Given a loss function $J(\theta)$, where θ represents the parameters of the model, the gradient descent algorithm iteratively updates the parameters in the direction of the negative gradient of the cost function with respect to the parameters. The update rule can be expressed as:

$$\theta := \theta - \alpha \nabla J(\theta), \tag{7}$$

where α is the learning rate, and $\nabla J(\theta)$ is the gradient of the cost function with respect to the parameters θ. The key idea behind gradient descent is to take small steps in the direction of the steepest descent of the cost function, which helps to find the optimal parameters that minimize the cost function.

Several variants of gradient descent, such as stochastic gradient descent (SGD) and mini-batch gradient descent(MBGD), have been developed to improve the efficiency of the algorithm. But the misuse of learning rate can lead to divergence or slow convergence. Techniques such as learning rate scheduling and momentum (e.g., Adam[1], AdamW[2]) have been developed to address these issues and improve the performance of gradient descent.

In the context of linear regression, the gradient of the cost function with respect to the parameters can be computed as:

$$\nabla J(\theta) = \nabla(\frac{1}{n}||\mathbf{y} - \mathbf{X}\theta||^2) = -\frac{2}{n} X^T(\mathbf{y} - \mathbf{X}\theta), \tag{8}$$

where $\mathbf{X}$ is the matrix of input features, $\mathbf{y}$ is the vector of observed values, and θ is the vector of parameters. By iteratively updating the parameters using the gradient descent algorithm, we can find the optimal parameters that minimize the cost function and improve the fit of

2.2.3 Newton's Method

Newton's method is an optimization algorithm used to find the roots of a function or to minimize a cost function in machine learning models, including linear regression. Given a cost function $J(\theta)$, the Newton's method updates the parameters using the second-order Taylor expansion:

$$\theta := \theta - (\nabla^2 J(\theta))^{-1} \nabla J(\theta), \tag{9}$$

where $\nabla J(\theta)$ is the gradient and $\nabla^2 J(\theta)$ is the Hessian matrix of the cost function.

3 Experiments

3.1 Baseline Model: Linear Regression

Table 1: Performance of three optimal algorithms on fitting linear regression.

Algorithm	Training		Testing	
	RMSE	R^2	RMSE	R^2
Least Squares	0.7832001	0.1412699	0.7669251	0.14288
Gradient Descent	0.7832001	0.1412699	0.7669251	0.14288
Newton's Method	0.7832001	0.1412699	0.7669251	0.14288

As shown in Table 1, the performance of three algorithms (least squares, gradient descent, and Newton's method) using metrics defined in Section 2.1. is nearly the same, which is expected since they are all solving the same optimization problem for linear regression. However, the RMSE values are relatively high and the R^2 values are low, indicating that the linear regression model does not fit the data well.

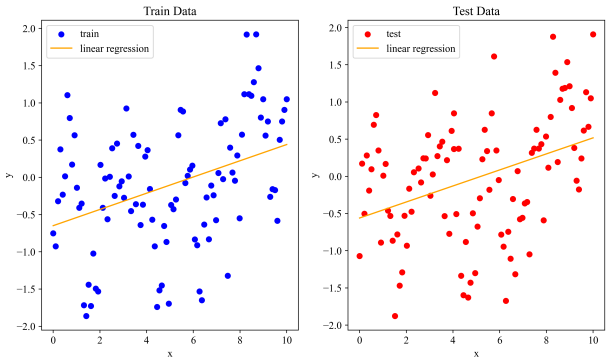

Figure 2: Linear regression fits on training and testing data.

Illustrated in Figure 2, the linear regression model fails to capture the nonlinear relationship between the input features and the output values, corresponding to the poor performance in metrics above.

3.2 model

4 Conclusion

References

[1] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[2] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. 2019. arXiv: 1711.05101 [cs.LG]. URL: https://arxiv.org/abs/1711.05101.